

AI Deep Learning: Recurrent Neural Networks Long Short-Term Memory (LSTM)

Thuan L Nguyen, PhD

Slide 2: Long Short-Term Memory Neural Networks (LSTM)



AI Deep learning (Source: mindovermachines.com)

Slide 3: Long Short-Term Memory Neural Networks (LSTM)

1. LSTM Neural Networks: Simple RNN with Short-Term Memory
2. LSTM Neural Networks : Introduction
3. LSTM Neural Networks : Core Concepts: Cell State & Gates: Overview
4. LSTM Neural Networks: Core Concepts: Cell State & Gates: Cell State
5. LSTM Neural Networks: Core Concepts: Cell State & Gates: Gates
6. LSTM Neural Networks: Flow of Information & Mathematical Models
7. LSTM Neural Networks: HOWTO Solve Vanishing Gradient Problem

Slide 4: Long Short-Term Memory Neural Networks (LSTM)

RNN: LSTM Cell: Cell (C) State and Gates: Flow of Information

In an LSTM cell, the flow of information occurs in 4 steps:

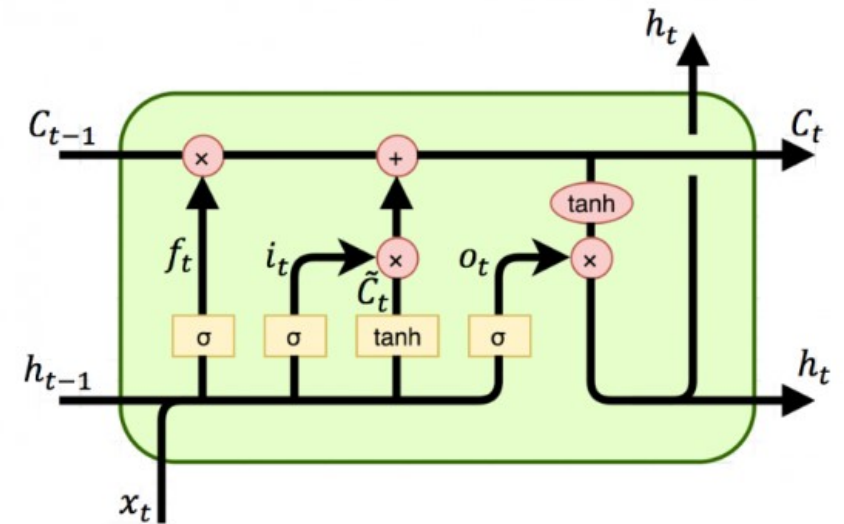
- Step 1: Forget gate
 - Decide **what information** of the **previous Cell state C_{t-1}** should be **removed** (irrelevant or insignificant) or **kept** (relevant or significant).

$$f(t) = \sigma(W_f * [h_{t-1}, x_t] + b_f)$$

- Step 2: Input Gate
 - Decide **what information** of the inputs (**h_{t-1}** and **x_t**) is **useful** and should be **added** into the **new current Cell state $C(t)$** .

$$i(t) = \sigma(W_i * [h_{t-1}, x_t] + b_i)$$

$$C(temp) = \tanh(W_c * [h_{t-1}, x_t] + b_c)$$



LSTM Cell (Source: Colah's Blog)

Slide 5: Long Short-Term Memory Neural Networks (LSTM)

RNN: LSTM Cell: Cell (C) State and Gates: Flow of Information

In an LSTM cell, the flow of information occurs in 4 steps:

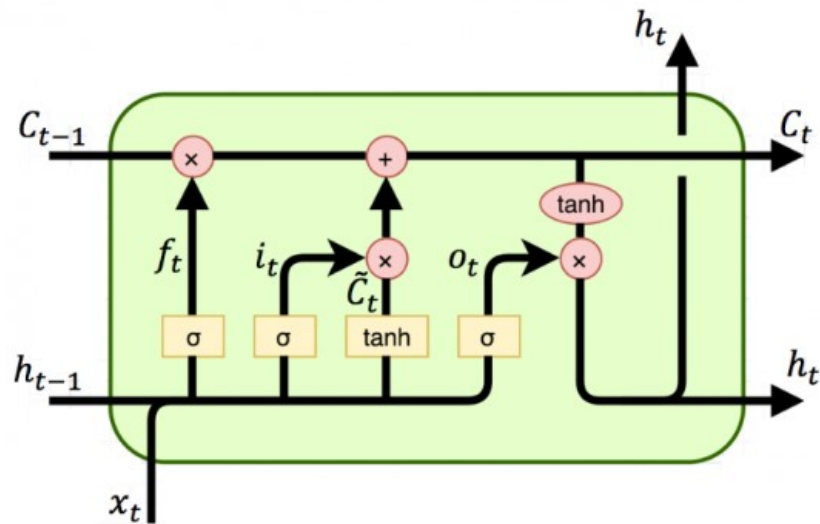
- Step 3: Output gate
 - Compute and extract the new current Cell state $\mathbf{C}(t)$ at the time step t .

$$C(t) = f_t * C_{t-1} + i_t * C(temp)$$

- Step 4: Output Gate
 - Decide what information of the inputs ($\mathbf{h}_{t-1}$ and $\mathbf{x}_t$) and the new current Cell state $\mathbf{C}(t)$ should be used to make the new current hidden state $\mathbf{h}(t)$ at the time step t .

$$o(t) = \sigma(W_o * [h_{t-1}, x_t] + b_o)$$

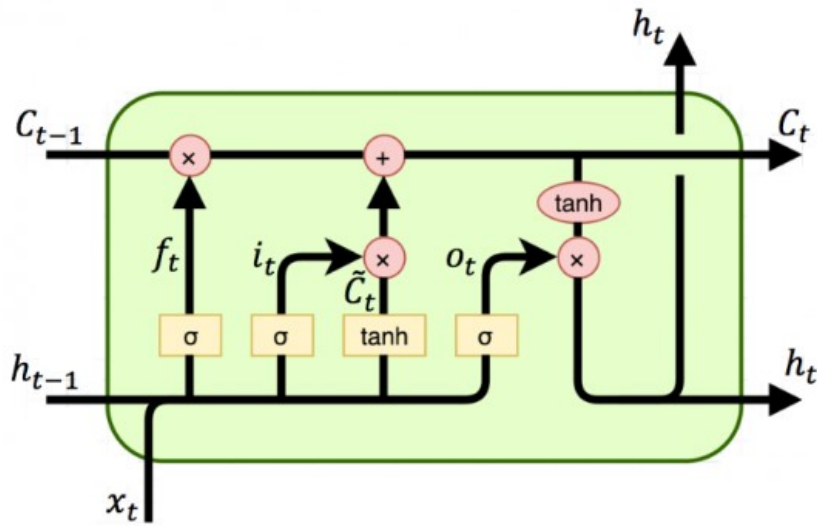
$$h(t) = o(t) * \tanh(C_t)$$



LSTM Cell (Source: Colah's Blog)

Slide 6: Long Short-Term Memory Neural Networks (LSTM)

RNN: LSTM Cell: Mathematical Models

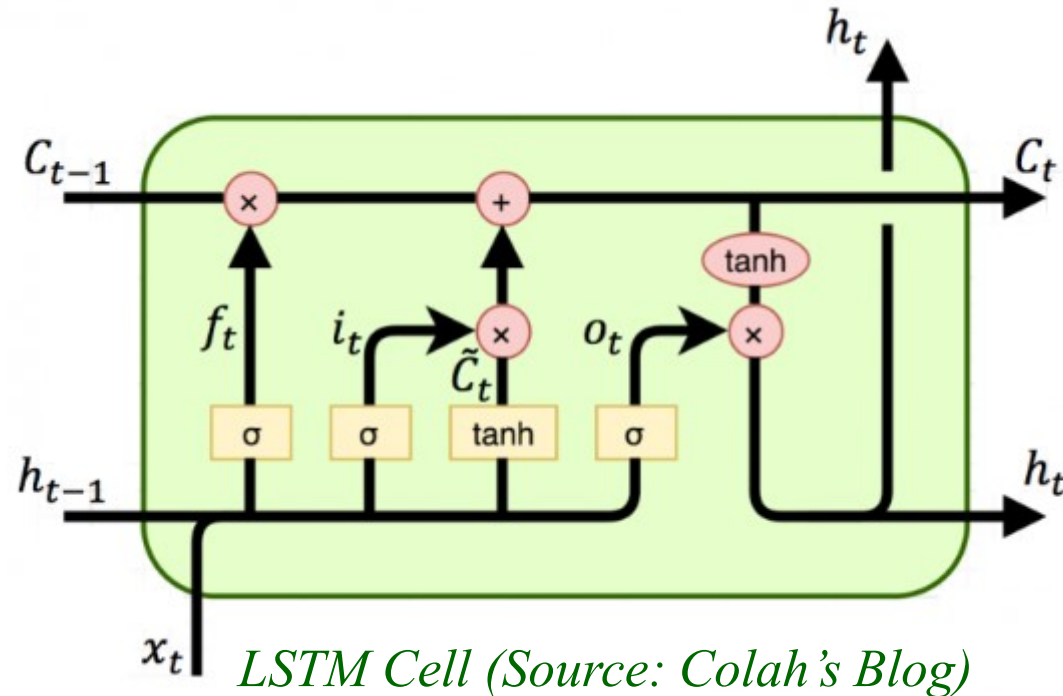


LSTM Cell (Source: Colah's Blog)

- $f(t) = \sigma(W_f * [h_{t-1}, x_t] + b_f)$
- $i(t) = \sigma(W_i * [h_{t-1}, x_t] + b_i)$
- $C(temp) = \tanh(W_c * [h_{t-1}, x_t] + b_c)$
- $C(t) = f_t * C_{t-1} + i_t * C(temp)$
- $o(t) = \sigma(W_o * [h_{t-1}, x_t] + b_o)$
- $h(t) = o(t) * \tanh(C_t)$

Slide 7: Long Short-Term Memory Neural Networks (LSTM)

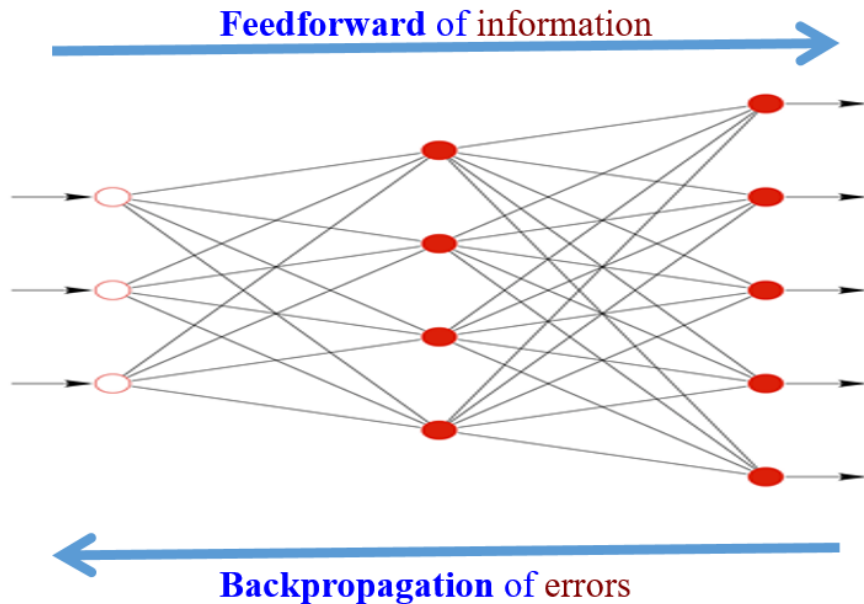
RNN: LSTM: Powerful Solution to Vanishing Gradient Problem



LSTM Neural Network: **HOWTO Solve** the **Vanishing Gradient Problem** ?

Slide 8: Long Short-Term Memory Neural Networks (LSTM)

Backpropagation: in Feedforward Neural Network (FFNN)



FFNN: Backpropagation

Scenario:

- It is assumed that the **desired output** of the network is y .
- Also assumed that the **neural network** predicted an **output y'** .
- The **difference** between the **predicted output** and the **desired output ($y' - y$)** is considered as the **cost** or **loss** represented by the **cost/loss function**.
 - The loss is high: The network makes a lot of mistakes
 - The loss is low: It makes fewer mistakes.

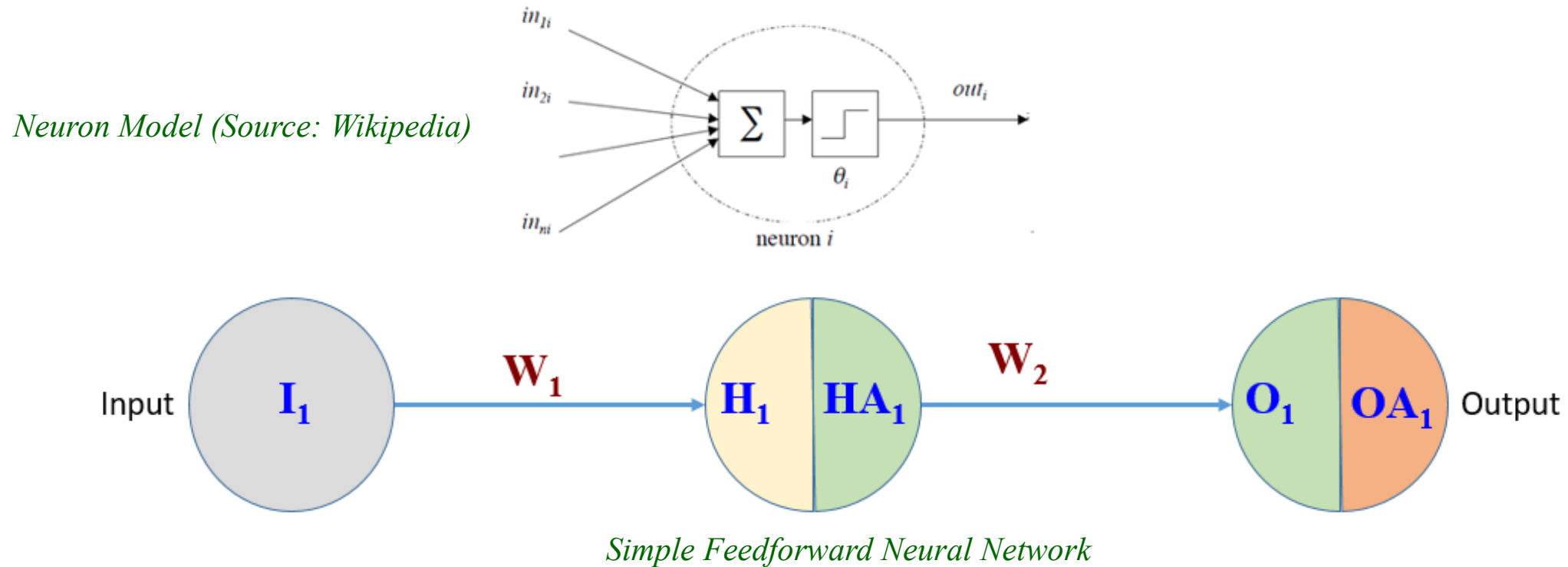
The **goal** of the **training/learning** process:

- To find the **weights (W)** and **bias (b)** that **minimizes** the **loss function E** over the training set.

→ Use **backpropagation** algorithm to **train** the neural network.

Slide 9: Long Short-Term Memory Neural Networks (LSTM)

Backpropagation: in Feedforward Neural Network (FFNN)

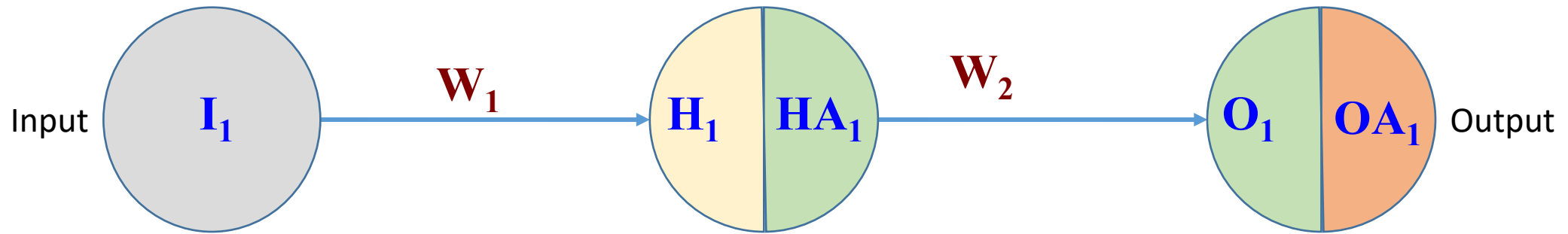


Given a very **simple** feedforward neural network (FFNN) that has **only three nodes**:

- > **Input** node: I_1
- > **Hidden** node: ($H_1 - HA_1$) representing a **hidden layer**
- > **Output** node: ($O_1 - OA_1$) representing the **output layer**

Slide 10: Long Short-Term Memory Neural Networks (LSTM)

Backpropagation: in Feedforward Neural Network (FFNN)

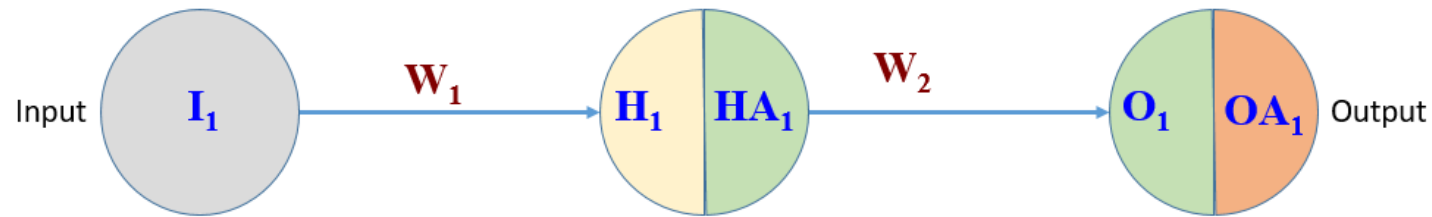


Simple Feedforward Neural Network

- **Af**: An activation function in a layer
- I_1 : Input 1
- $H_1 = W_1 * I_1$: Value of Node 1 before being processed by the Af
- HA_1 : Output of Node 1: Value of Node 1 after being processed by the Af
- $O_1 = W_2 * HA_1$: Value of Output node before being processed by the Af
- OA_1 : Output

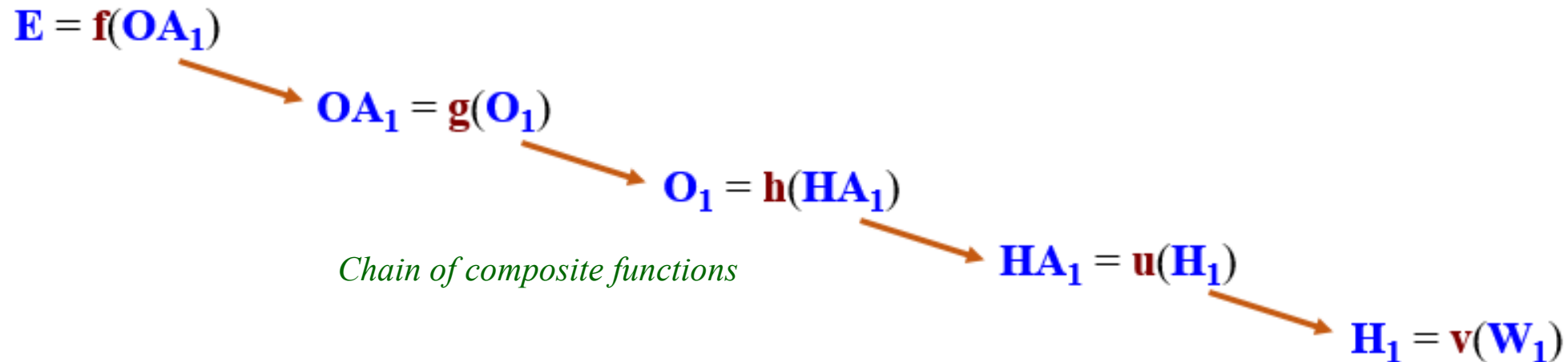
Slide 11: Long Short-Term Memory Neural Networks (LSTM)

Backpropagation: in Feedforward Neural Network (FFNN)



Simple Feedforward Neural Network

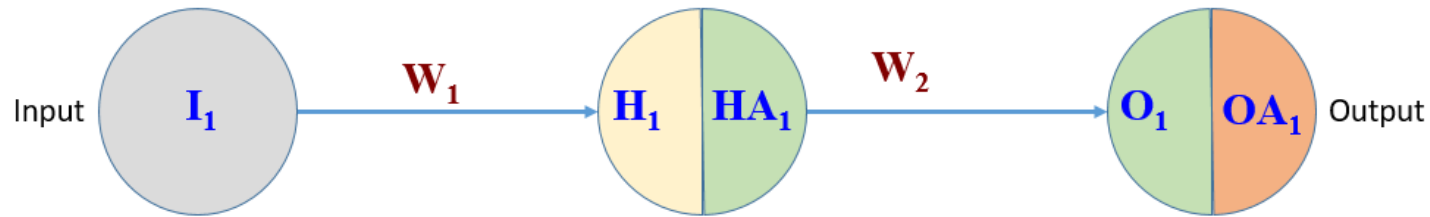
- E : Error, Cost, Loss
- OA_1 : Output
- $E = \text{Real Value} - OA_1$
- $E = f(OA_1)$



Chain of composite functions

Slide 12: Long Short-Term Memory Neural Networks (LSTM)

Backpropagation: in Feedforward Neural Network (FFNN)



Simple Feedforward Neural Network

- **E**: Error, Cost, Loss
- **OA₁**: Output
- **E** = Real Value – **OA₁**
- **E** = **f**(**OA₁**)

$$\mathbf{E} = \mathbf{f}(\mathbf{OA}_1, \dots)$$

$$\mathbf{OA}_1 = \mathbf{g}(\mathbf{O}_1, \dots)$$

$$\mathbf{O}_1 = \mathbf{h}(\mathbf{HA}_1, \dots)$$

$$\mathbf{HA}_1 = \mathbf{u}(\mathbf{H}_1, \dots)$$

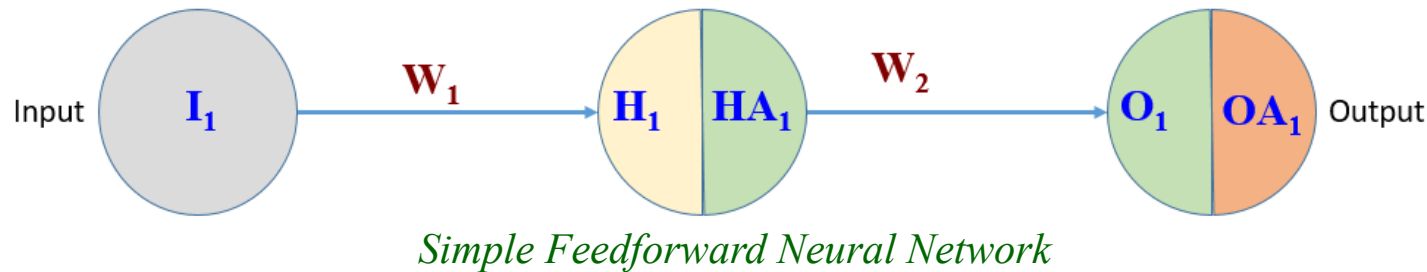
$$\mathbf{H}_1 = \mathbf{v}(\mathbf{W}_1, \dots)$$

Chain of composite functions

NOTES: *f, g, h, u, v are multivariable functions (in general case).*

Slide 13: Long Short-Term Memory Neural Networks (LSTM)

Backpropagation: in Feedforward Neural Network (FFNN)



E: Error, Cost, Loss

OA_1 : Output

E = Real Value – **OA_1**

E = **f**(**OA_1**)

What is a **Gradient**?

- "A gradient measures **how much the output of a function changes** if you **change the inputs a little bit**." - Lex Fridman (MIT)
- For a machine learning or **deep learning model**, a gradient measures the **change in all weights** with regard to the **change in cost or error**.

What is a **Derivative** of a **function $y = f(x)$** ?

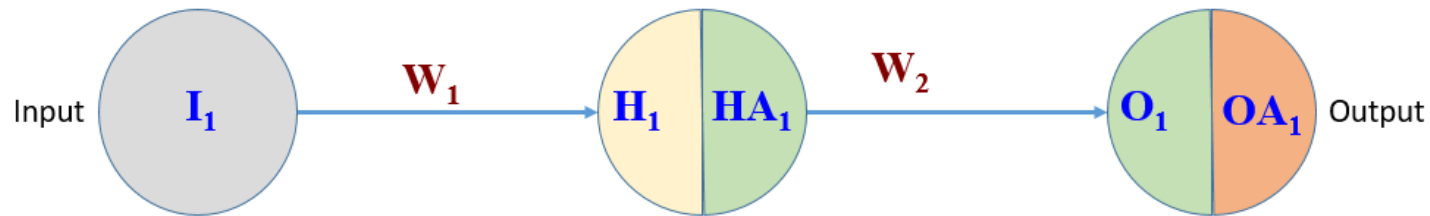
Derivative of **y** with respect to (w.r.t.) **x**:

- A **measure** of the **rate** at which the **value y** of the function **changes** with respect to the **change** of the **variable x**.
- Is called the **derivative** of **f** with respect to **x**.
- Denoted by **$f'(x) = \frac{df}{dx} = \frac{dy}{dx}$**

→ **Gradient** is the **derivative** of the **loss function w.r.t. weight**.

Slide 14: Long Short-Term Memory Neural Networks (LSTM)

Backpropagation: in Feedforward Neural Network (FFNN)



Simple Feedforward Neural Network

E: Error, Cost, Loss

OA₁: Output

E = Real Value – **OA₁**

E = **f**(**OA₁**)

Composite Functions: **Derivatives** and **Chain Rule**

Given: Functions $y = f(u)$, $u = g(v)$, and $v = h(x)$

$y = f(u) = f(g(v)) = f(g(h(x))) = (f \circ g \circ h)(x)$

y is a **composite** function.

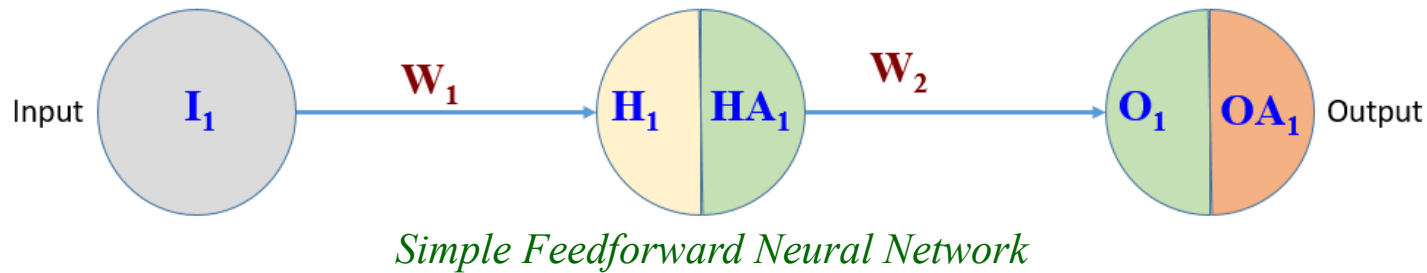
Applying **chain rule** $\rightarrow$ **Derivative** of y :

$$\frac{d}{dx} f(g(h(x))) = f'(g(h(x))) * g'(h(x)) * h'(x)$$

$$f'(x) = y'(x) = \frac{dy}{dx} = \frac{dy}{du} * \frac{du}{dv} * \frac{dv}{dx}$$

Slide 15: Long Short-Term Memory Neural Networks (LSTM)

Backpropagation: in Feedforward Neural Network (FFNN)



E : Error, Cost, Loss

OA_1 : Output

$E = \text{Real Value} - OA_1$

$E = f(OA_1)$

Given: a multivariable function $f(x, y)$ that has two independent variables x and y .

Partial derivative of $z = f(x, y)$ w.r.t. x :

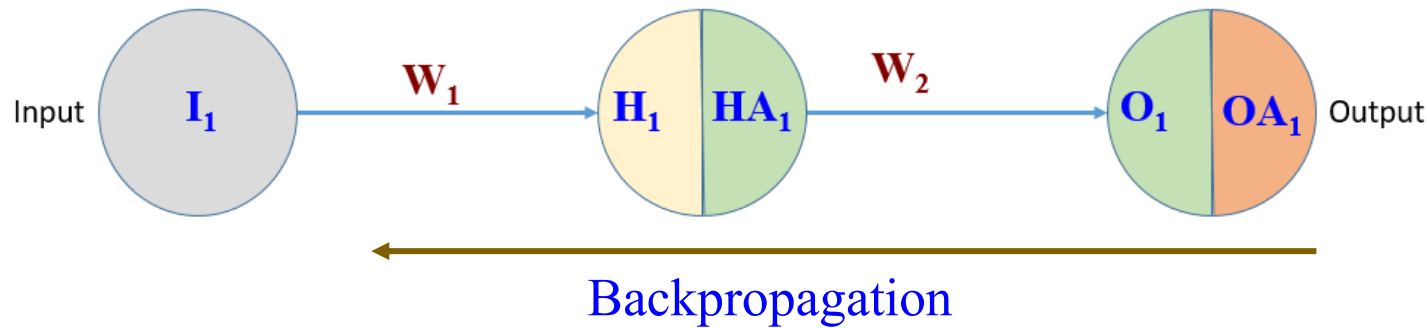
- A **measure** of the **rate** at which the **value** Z of the function **changes** with respect to the **change** of the variable x , **with** y held as **constant**.

- Is called the **partial derivative** of f with respect to x .

- Denoted by $f'_x = \frac{\partial f}{\partial x} = \frac{\partial z}{\partial x}$

Slide 16: Long Short-Term Memory Neural Networks (LSTM)

Backpropagation: Gradient Computation (in FFNN)



E: Error, Cost, Loss

OA₁: Output

E = Real Value – **OA₁**

E = **f**(**OA₁**)

Backward Pass

- The **difference** between the **predicted output** and the **actual value** of outcome, i.e., **error, cost, or lost E**, is used to **change** the **weights (W)** of the neurons at each layer recursively such that the **error decreases gradually**.

Updated Weight

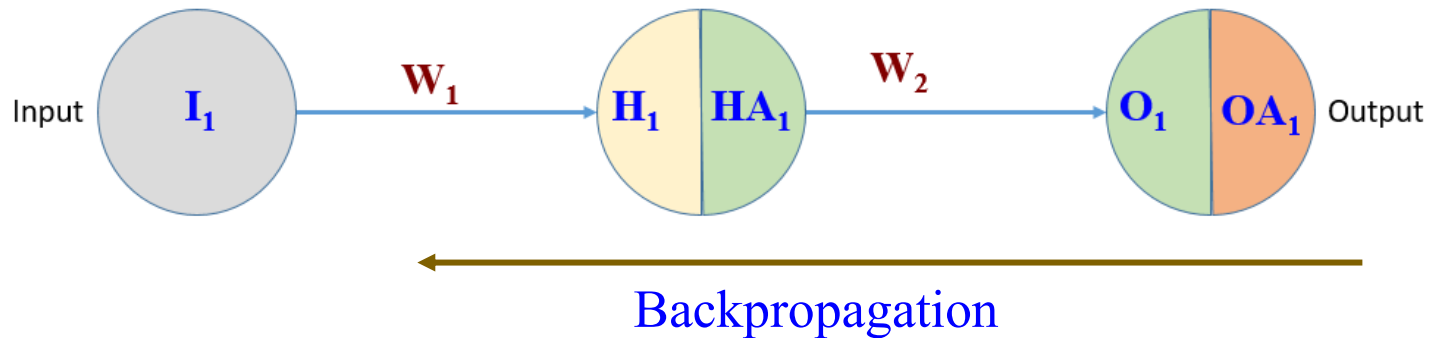
GRADIENT

$$W \leftarrow W - \alpha \frac{\partial E}{\partial W}$$

Current Weight

Slide 17: Long Short-Term Memory Neural Networks (LSTM)

Backpropagation: Gradient Computation (in FFNN)



E : Error, Cost, Loss

OA_1 : Output

$E = \text{Real Value} - OA_1$

$E = f(OA_1)$

The **goal** of the backpropagation algorithm:

- To find the **weights** (W) and **bias** (b) that **minimize** the loss function E over the training set.

→ Compute the **gradient** to **update** the weights and biases

Updated Weight

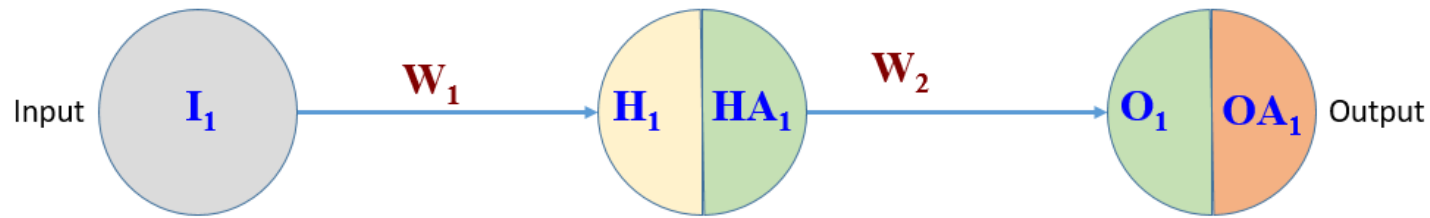
GRADIENT

$$W \leftarrow W - \alpha \frac{\partial E}{\partial W}$$

Current Weight

Slide 18: Long Short-Term Memory Neural Networks (LSTM)

Backpropagation: Gradient Computation (in FFNN)



Simple Feedforward Neural Network

E: Error, Cost, Loss

OA₁: Output

E = Real Value – **OA₁**

E = **f**(**OA₁**)

$$\mathbf{E} = \mathbf{f}(\mathbf{OA}_1, \dots)$$

$$\mathbf{OA}_1 = \mathbf{g}(\mathbf{O}_1, \dots)$$

Chain of composite functions

$$\mathbf{O}_1 = \mathbf{h}(\mathbf{HA}_1, \dots)$$

$$\mathbf{HA}_1 = \mathbf{u}(\mathbf{H}_1, \dots)$$

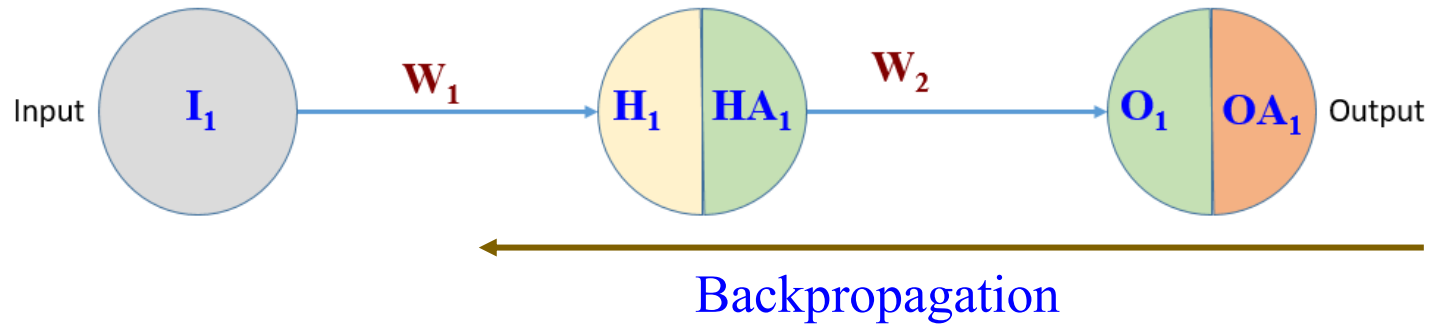
$$\mathbf{H}_1 = \mathbf{v}(\mathbf{W}_1, \dots)$$

$$\mathbf{E} = (\mathbf{f} \circ \mathbf{g} \circ \mathbf{h} \circ \mathbf{u} \circ \mathbf{v})(\mathbf{W}_1)$$

NOTES: ***f**, **g**, **h**, **u**, **v** are multivariable functions (in general case).*

Slide 19: Long Short-Term Memory Neural Networks (LSTM)

Backpropagation: Gradient Computation (in FFNN)



E : Error, Cost, Loss

OA_1 : Output

$E = \text{Real Value} - OA_1$

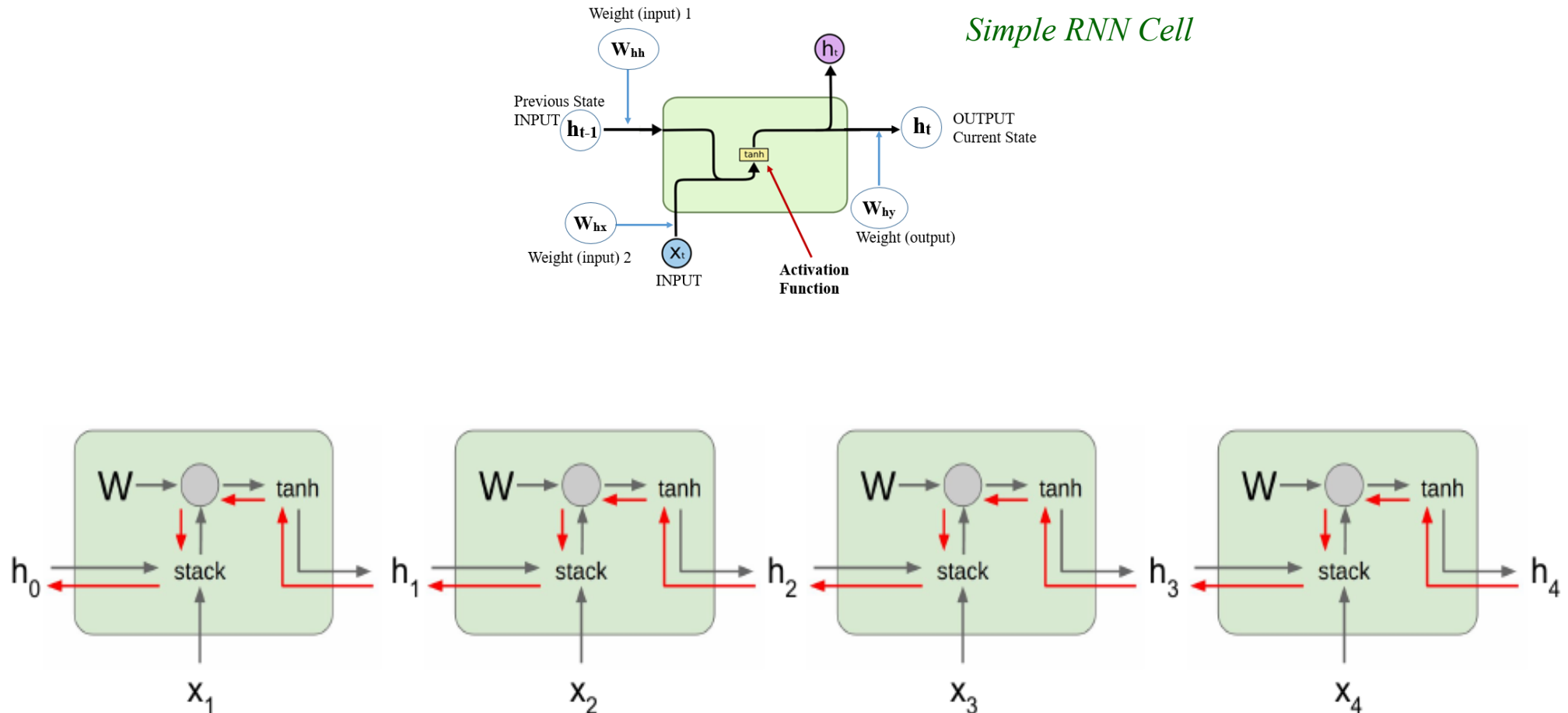
$E = f(OA_1)$

$$E = (f \circ g \circ h \circ u \circ v)(W_1)$$

$$\frac{\partial E}{\partial W_1} = \frac{\partial E}{\partial OA_1} * \frac{\partial OA_1}{\partial O_1} * \frac{\partial O_1}{\partial HA_1} * \frac{\partial HA_1}{\partial H_1} * \frac{\partial H_1}{\partial W_1}$$

Slide 20: Long Short-Term Memory Neural Networks (LSTM)

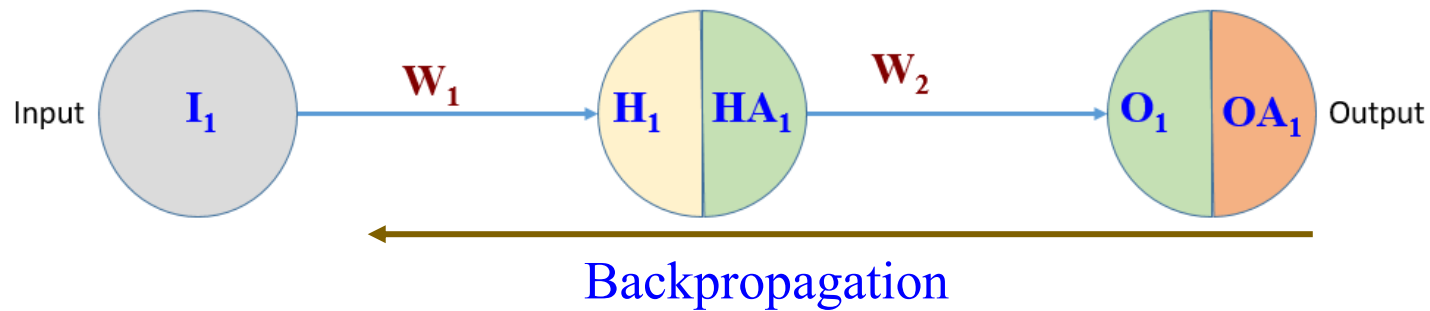
Backpropagation Through Time (BPTT) (in Simple RNN)



BPTT in SimpleRNN (Source: Stanford.edu; Bengio et al., 1994; Pascanu et al., 2013)

Slide 21: Long Short-Term Memory Neural Networks (LSTM)

Backpropagation Through Time (BPTT) (in Simple RNN)

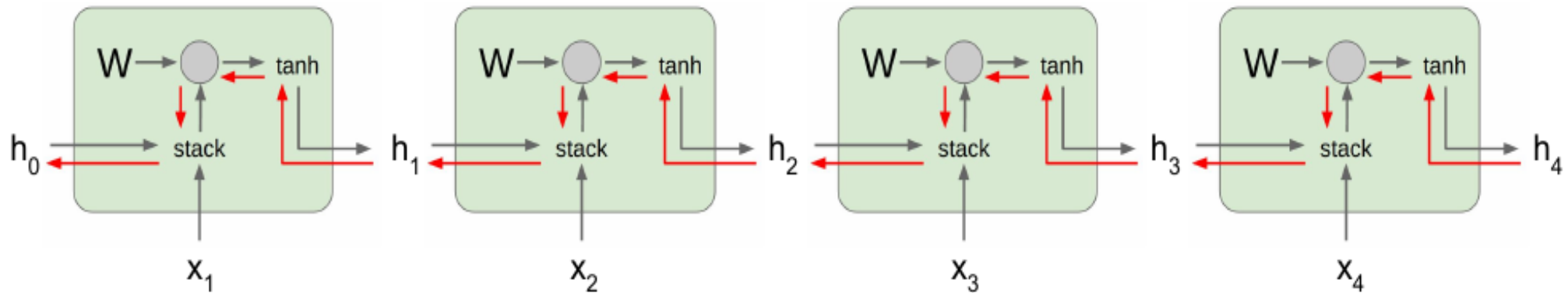


E: Error, Cost, Loss

OA₁: Output

E = Real Value – **OA₁**

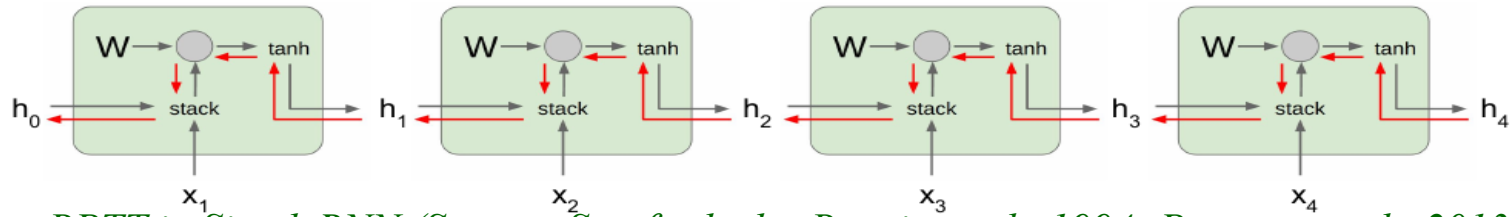
E = **f**(**OA₁**)



BPTT in SimpleRNN (Source: Stanford.edu; Bengio et al., 1994; Pascanu et al., 2013)

Slide 22: Long Short-Term Memory Neural Networks (LSTM)

BPTT: Gradient Computation (in Simple RNN)



BPTT in SimpleRNN (Source: Stanford.edu; Bengio et al., 1994; Pascanu et al., 2013)

- Given a **simple RNN** with a **learning task** of **T** time steps.
- The **gradient** of the error at the **time step k** can be computed as:

$$h_t = f_W(h_{t-1}, x_t)$$

$$\frac{\partial E_k}{\partial W} = \frac{\partial E_k}{\partial h_k} * \frac{\partial h_k}{\partial h_{k-1}} * \frac{\partial h_{k-1}}{\partial h_{k-2}} * \frac{\partial h_{k-2}}{\partial h_{k-3}} * \dots * \frac{\partial h_2}{\partial h_1} * \frac{\partial h_1}{\partial W}$$

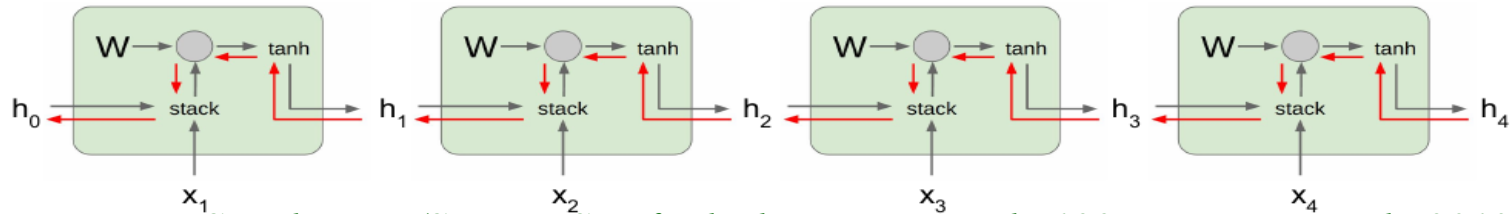
$$\frac{\partial E_k}{\partial W} = \frac{\partial E_k}{\partial h_k} * \frac{\partial h_k}{\partial h_{k-1}} * \left(\prod_{t=2}^k \frac{\partial h_t}{\partial h_{t-1}} \right) * \frac{\partial h_1}{\partial W}$$

$$h_t = \tanh(W_{hh} * h_{t-1} + W_{hx} * x_t)$$

$$\frac{\partial h_t}{\partial h_{t-1}} = h'(t) = \dots = \tanh'(W_{hh} * h_{t-1} + W_{hx} * x_t) * W_{hh}$$

Slide 23: Long Short-Term Memory Neural Networks (LSTM)

BPTT: Gradient Computation (in Simple RNN)



BPTT in SimpleRNN (Source: Stanford.edu; Bengio et al., 1994; Pascanu et al., 2013)

$$\frac{\partial E_k}{\partial W} = \frac{\partial E_k}{\partial h_k} * \frac{\partial h_k}{\partial h_{k-1}} * \left(\prod_{t=2}^k \frac{\partial h_t}{\partial h_{t-1}} \right) * \frac{\partial h_1}{\partial W}$$

$$\frac{\partial h_t}{\partial h_{t-1}} = h'(t) = \dots = \tanh'(W_{hh} * h_{t-1} + W_{hx} * x_t) * W_{hh}$$

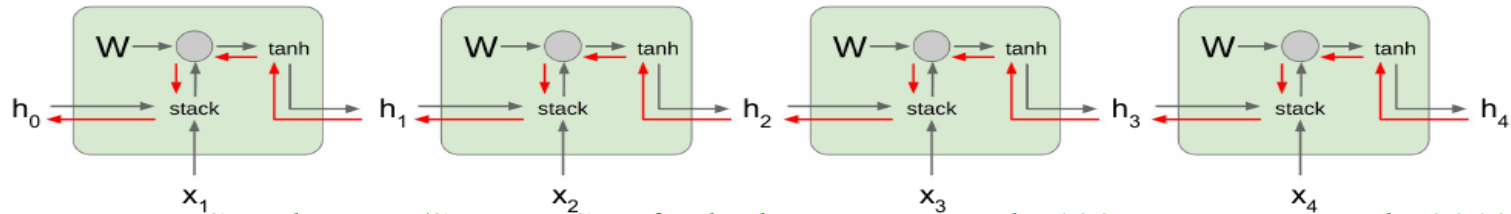
Case 1: W_{hh} is not very large (very often)

→ NOT overpower $\tanh'(\dots)$ → **Vanishing Gradient:**

- $\tanh'$ (derivative of $\tanh$) < 1
- So: $\prod_{t=2}^k \frac{\partial h_t}{\partial h_{t-1}} \rightarrow 0$ when k is large, i.e., long sequence
- So: $\frac{\partial E_k}{\partial W} \rightarrow 0$: **Gradient is vanishing.**
- W is **barely updated** → The network **stop learning**.

Slide 24: Long Short-Term Memory Neural Networks (LSTM)

BPTT: Gradient Computation (in Simple RNN)



BPTT in SimpleRNN (Source: Stanford.edu; Bengio et al., 1994; Pascanu et al., 2013)

$$\frac{\partial E_k}{\partial W} = \frac{\partial E_k}{\partial h_k} * \frac{\partial h_k}{\partial h_{k-1}} * \left(\prod_{t=2}^k \frac{\partial h_t}{\partial h_{t-1}} \right) * \frac{\partial h_1}{\partial W}$$

$$\frac{\partial h_t}{\partial h_{t-1}} = \mathbf{h}'(t) = \dots = \tanh'(\mathbf{W}_{hh} * \mathbf{h}_{t-1} + \mathbf{W}_{hx} * \mathbf{x}_t) * \mathbf{W}_{hh}$$

Case 2: $\mathbf{W}_{hh}$ is very large (not very often)

→ Large enough to overpower $\tanh'(\dots)$

→ Exploding Gradient:

- $\tanh'$ (derivative of $\tanh$) < 1 BUT $\mathbf{W}_{hh}$ is so large.
- So: $\prod_{t=2}^k \frac{\partial h_t}{\partial h_{t-1}}$ increases quickly when k is large
- So: $\frac{\partial E_k}{\partial W}$ increases quickly: Gradient is exploding.
- The network becomes unstable and unable to learn.

Slide 25: Long Short-Term Memory Neural Networks (LSTM)

Backpropagation Through Time (BPTT): in LSTM

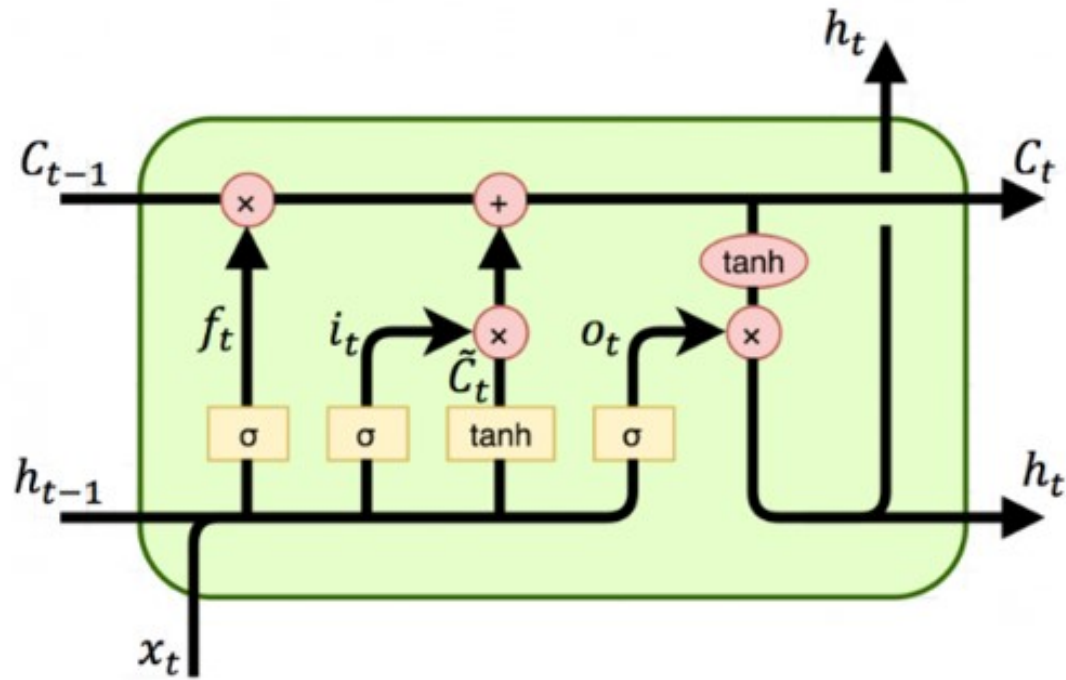


Photo Source: Colah's Blog

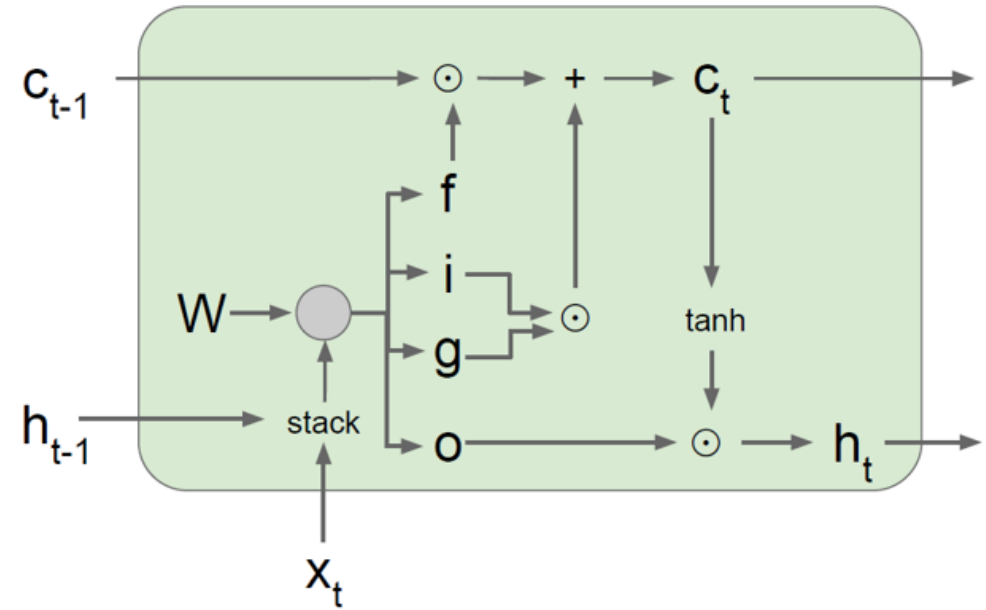


Photo Source: Stanford.edu; Hochreiter et al., 1997.

LSTM Cell

Slide 26: Long Short-Term Memory Neural Networks (LSTM)

Backpropagation Through Time (BPTT): in LSTM

Uninterrupted gradient flow!

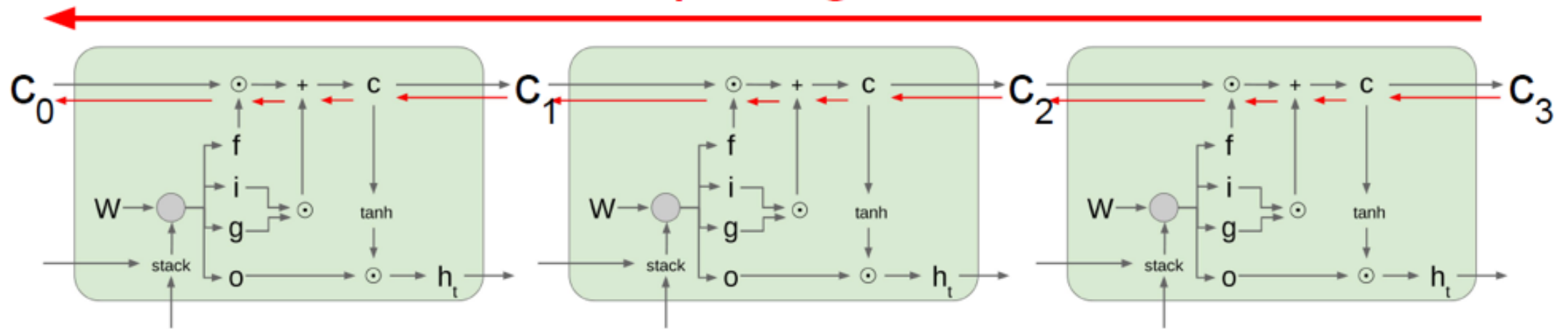


Photo Source: Stanford.edu; Hochreiter et al., 1997.

Slide 27: Long Short-Term Memory Neural Networks (LSTM)

Backpropagation Through Time (BPTT): in LSTM

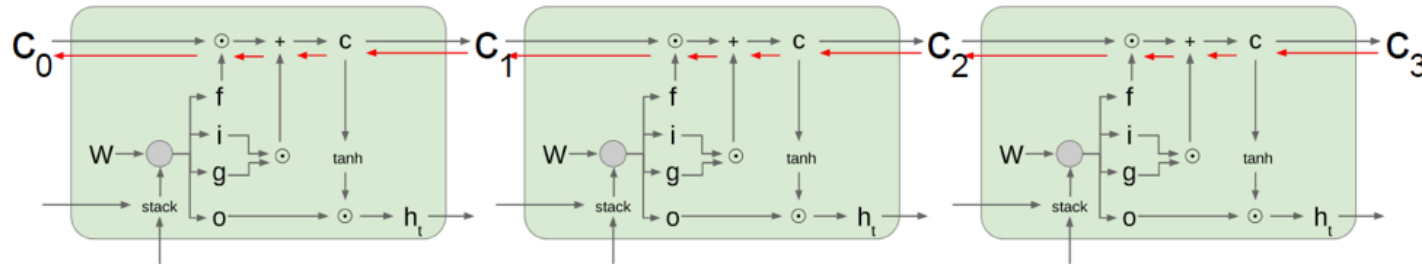


Photo Source: Stanford.edu; Hochreiter et al., 1997.

LSTM: BPTT: Flow of gradient

- Given an LSTM neural network with a learning task of T time steps.
- The **gradient** of the error at the **time step k** can be computed as:

$$\frac{\partial E_k}{\partial W} = \frac{\partial E_k}{\partial C_k} * \frac{\partial C_k}{\partial C_{k-1}} * \frac{\partial C_{k-1}}{\partial C_{k-2}} * \frac{\partial C_{k-2}}{\partial C_{k-3}} * \dots * \frac{\partial C_2}{\partial C_1} * \frac{\partial C_1}{\partial W}$$

$$\frac{\partial E_k}{\partial W} = \frac{\partial E_k}{\partial C_k} * \frac{\partial C_k}{\partial C_{k-1}} * \left(\prod_{t=2}^k \frac{\partial C_t}{\partial C_{t-1}} \right) * \frac{\partial C_1}{\partial W}$$

Slide 28: Long Short-Term Memory Neural Networks (LSTM)

Backpropagation Through Time (BPTT): in LSTM

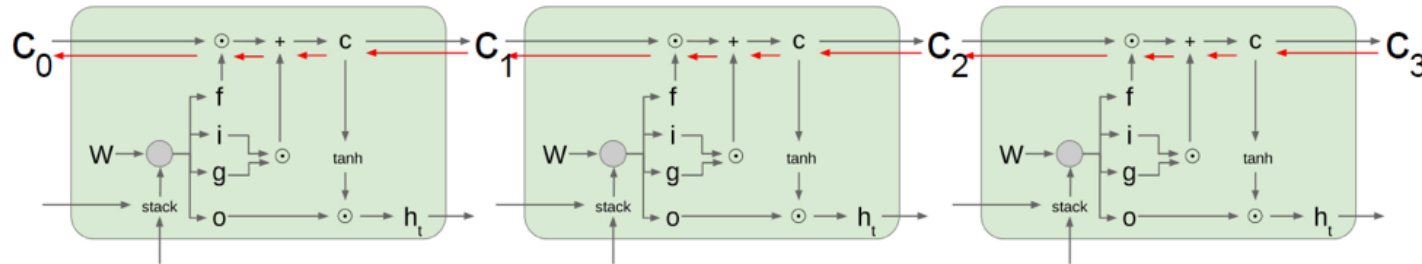


Photo Source: Stanford.edu; Hochreiter et al., 1997.

LSTM: BPTT: Flow of gradient

- Given an LSTM neural network with a learning task of T time steps.
- The **gradient** of the error at the **time step k** can be computed as:

$$\frac{\partial E_k}{\partial W} = \frac{\partial E_k}{\partial C_k} * \frac{\partial C_k}{\partial C_{k-1}} * \left(\prod_{t=2}^k \frac{\partial C_t}{\partial C_{t-1}} \right) * \frac{\partial C_1}{\partial W}$$

$$\frac{\partial C_t}{\partial C_{t-1}} = \dots = \dots = \dots = \mathbf{f}(t) + \frac{d}{dC_{t-1}} \left(\tanh(W_c \cdot [h_{t-1}, X_t]) * \sigma(\tanh(W_i \cdot [h_{t-1}, X_t])) \right)$$

Slide 29: Long Short-Term Memory Neural Networks (LSTM)

Backpropagation Through Time (BPTT): in LSTM

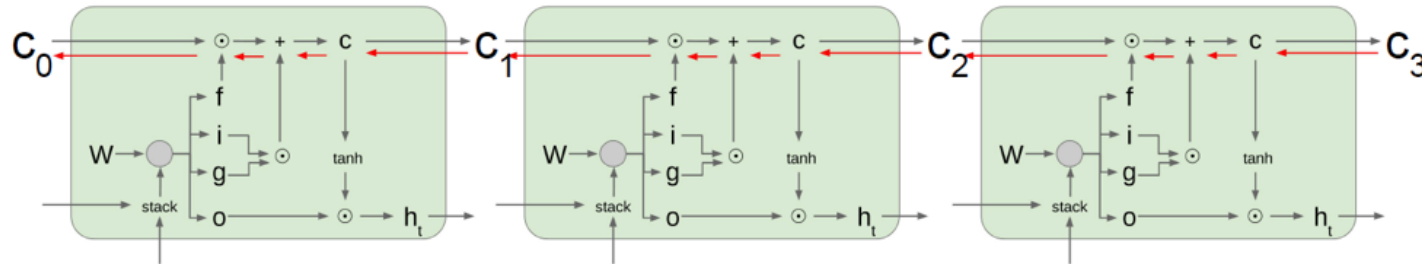


Photo Source: Stanford.edu; Hochreiter et al., 1997.

LSTM: BPTT: Flow of gradient

- Given an LSTM neural network with a learning task of **T** time steps.
- The **gradient** of the error at the **time step k** can be computed as:

$$\frac{\partial C_t}{\partial C_{t-1}} = \dots = \dots = \dots = \mathbf{f}(t) + \frac{d}{dC_{t-1}} (\tanh (W_c \cdot [h_{t-1}, X_t]) * \sigma(\tanh(W_i \cdot [h_{t-1}, X_t])))$$

In an **LSTM cell** at a **time step t**:

- If the **forget gate** decides to **keep some information** from the previous Cell (C) state
 - The **value of f(t)** should be ≈ 1 or **1**.

→ $\frac{\partial C_t}{\partial C_{t-1}}$: **not shrunk** to 0

Slide 30: Long Short-Term Memory Neural Networks (LSTM)

RNN: LSTM: Powerful Solution to Vanishing Gradient Problem

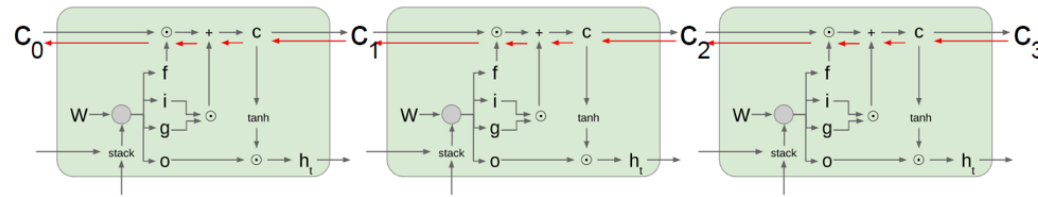


Photo Source: Stanford.edu; Hochreiter et al., 1997.

LSTM: BPTT: Flow of gradient

- Given an LSTM neural network with a learning task of **T** time steps.
- The **gradient** of the error at the **time step k** can be computed as:

$$\frac{\partial C_t}{\partial C_{t-1}} = \dots = \mathbf{f}(t) + \frac{d}{dC_{t-1}} (\tanh (W_c \cdot [h_{t-1}, X_t]) * \sigma(\tanh(W_i \cdot [h_{t-1}, X_t])))$$

$$\frac{\partial E_k}{\partial W} = \frac{\partial E_k}{\partial C_k} * \frac{\partial C_k}{\partial C_{k-1}} * \left(\prod_{t=2}^k \frac{\partial C_t}{\partial C_{t-1}} \right) * \frac{\partial C_1}{\partial W}$$

$$\frac{\partial C_t}{\partial C_{t-1}} : \text{not shrunk to 0}$$

$$\rightarrow \frac{\partial E_k}{\partial W} : \text{not shrunk to 0}$$

→ **NO** vanishing gradient